

SMART INDIA HACKATHON 2025

EDGE FLEET



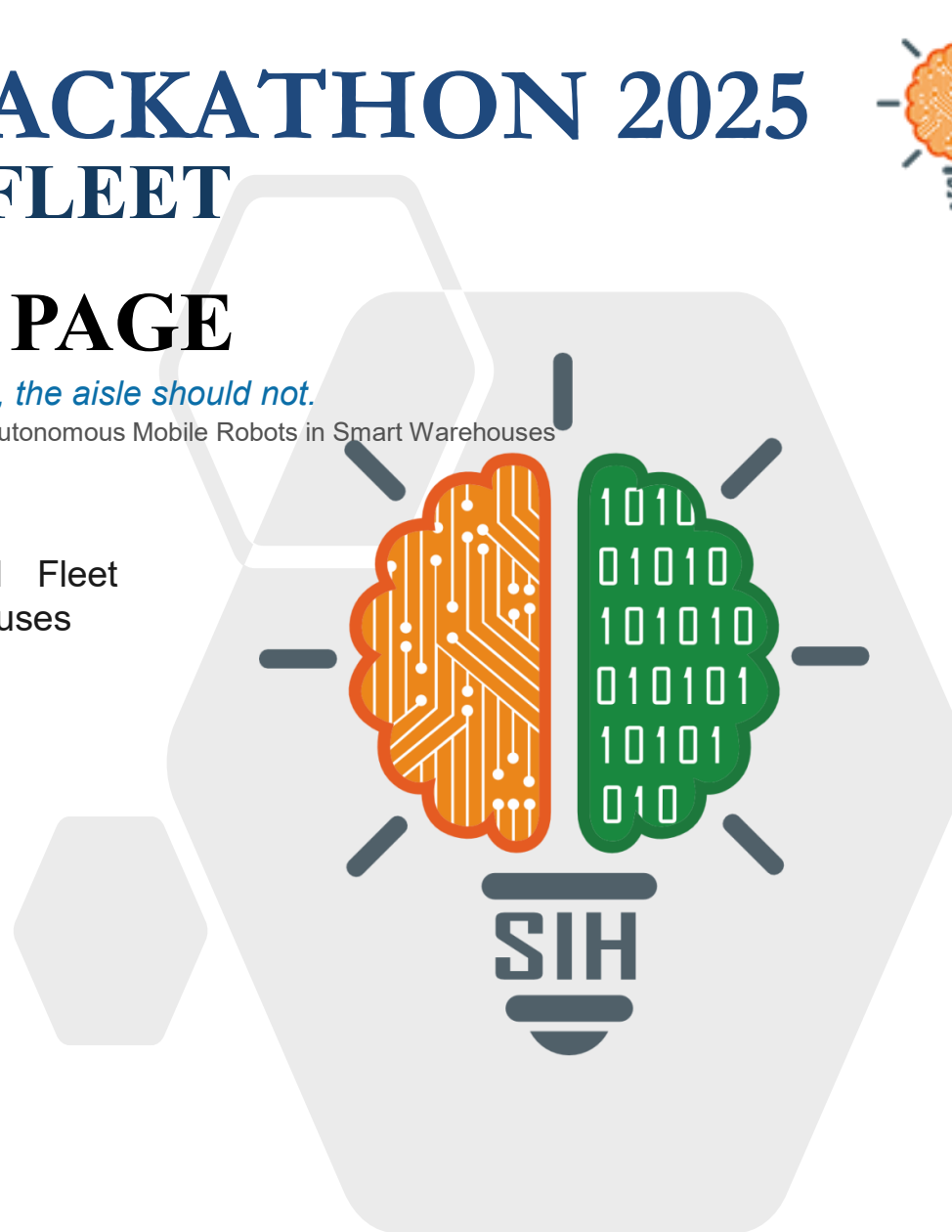
TITLE PAGE

If the control room dies, the aisle should not.

Edge-AI Based Distributed Fleet Coordination for Autonomous Mobile Robots in Smart Warehouses

- Problem Statement ID – SIH26123
- Problem Statement Title – Edge-AI Based Distributed Fleet Coordination for Autonomous Mobile Robots in Smart Warehouses
- Theme – Robotics and Drones
- PS Category – Software
- **Team ID – «Team ID from portal»**
- **Team Name (Registered on portal) – «Team Name»**

- 3 AMRs · no dispatcher · peer-to-peer radio only. The dashboard is a spectator, never a controller.
- Measured at a 800-tick limit: stop-and-wait never completes an experiment; UBPA finishes E1 in 72 ticks, 6/6 tasks, 0 deadlocks.
- Python 3.10+ standard library only — no pip install, no build step, no GPU, no cloud.





IDEA TITLE



EDGE FLEET — robots that negotiate the aisle instead of waiting for a scheduler

If the control room dies, the aisle should not. Three warehouse AMRs with no central dispatcher: each robot broadcasts its own state, bids for work, detects conflict on a peer's plan and yields by an explainable rule — so the fleet keeps moving when one robot, or the radio, misbehaves.

Proposed Solution (Describe your Idea/Solution/Prototype)

A Python 3.10+ standard-library prototype: 3+ autonomous AMRs on a Hyderabad 3PL warehouse grid, each running its own space-time A* planner and its own auction. One terminal is the clock (8 Hz); the browser dashboard is a spectator, never a controller.

Detailed explanation of the proposed solution

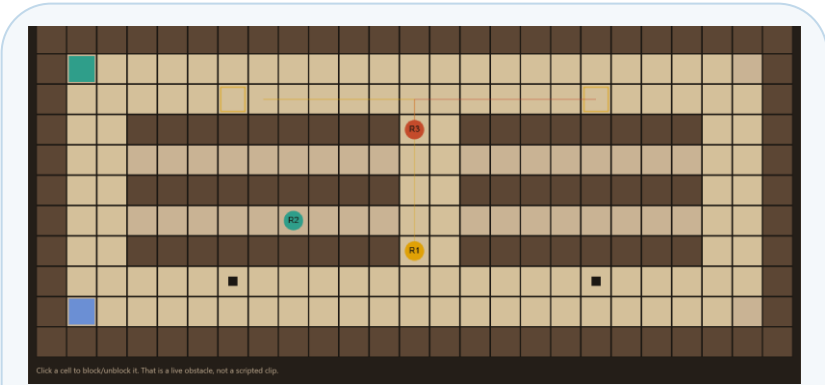
- Every tick each robot broadcasts PeerState — robot_id, position, velocity, current_task, planned_path, battery, priority, timestamp, status. That packet is the only channel between robots.
- Work is allocated by auction, not assignment: cost = distance + battery penalty + congestion + workload, and the lowest valid bid wins by each robot's own argmin. Tie-break on robot_id, so no referee exists.
- Conflict is detected from a peer's *broadcast* planned_path — never a central occupancy oracle — then settled locally with YIELD_QUERY / YIELD_ACK / YIELD_NACK.
- UBPA right-of-way picks who keeps the aisle; the loser reverse-plans to the nearest turnout, waits, and resumes.
- Block a cell live and the sensing robot broadcasts OBSTACLE; if a pick becomes unreachable the holder RELEASEs it and the peers re-auction.

How it addresses the problem

- Warehouse fleets are normally coordinated by one dispatcher — a single point of failure and a scaling ceiling. Put the decision inside each robot and the aisle survives the control room.
- Degraded radio is measured, not assumed (results/degradation.json): at 40% packet loss the same 6/6 jobs still finish with 0 deadlocks — just later (93–125 ticks vs 72 clean).

Innovation and uniqueness of the solution

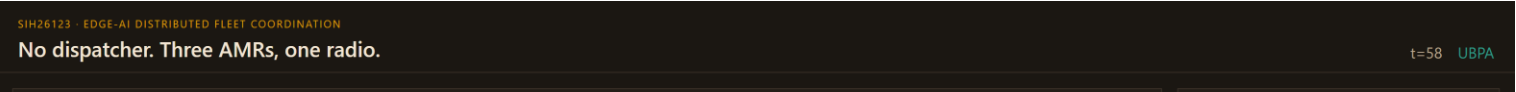
- UBPA is our own rule — score = $4.0 \times \text{task_urgency} + 2.0 \times \text{battery}/100 + 1.5 \times \text{priority} + 1.0 \times \text{arrival_bonus}$; the lower score yields to a turnout. A stalemate timeout on the lower robot_id is the safety net, not the rule.
- Decentralization is *tested*, not promised: 9 unit tests fail the build if a central scheduler or a hardcoded demo winner appears.
- Nothing is inflated — improvement is null in metrics.json because the baseline never finished, so this deck quotes raw values only.

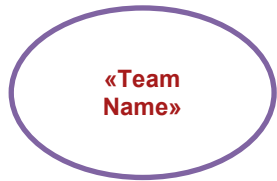


PROOF — the live floor at tick 44: three AMRs, their planned paths, and 3 of 6 tasks already done with 0 deadlocks. Captured from the running dashboard.

The three questions this PS is judged on

- Who assigns tasks? Nobody — each robot bids and independently checks who won.
- Who has right of way? The UBPA score, then robot_id.
- What if both peers hold stale state? After 8 stalemate ticks the lower robot_id yields. No central lock is ever taken.





TECHNICAL APPROACH



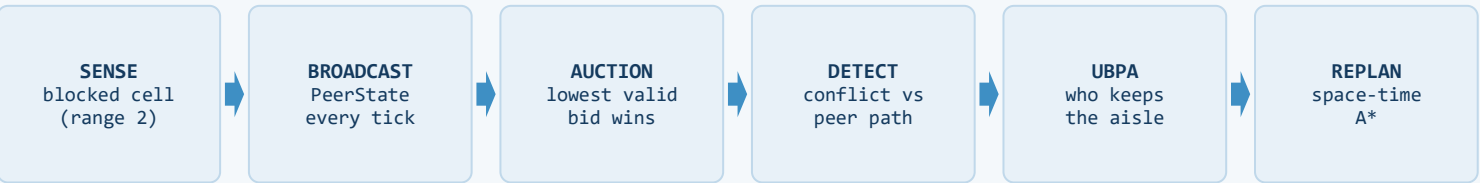
Technologies to be used (e.g. programming languages, frameworks, hardware)

Python 3.10+ standard library only — no pip install, no build step, no GPU, no cloud, no service to configure. Dashboard: vanilla HTML/CSS/JS on stdlib http.server. Storage: JSON (results/metrics.json). Radio: an in-process peer bus with drop probability and latency. Roadmap hardware: one Raspberry-Pi-class edge computer per AMR, radio becomes UDP multicast, and the same Robot.step() runs unchanged.

Methodology and process for implementation (Flow Charts/Images/ working prototype)

- Local planner: space-time A* on a 4-connected 25 × 11 grid; state (x, y, t); actions N/E/S/W/WAIT; peer planned_path cells become reservations; unknown future is held for HOLD_HORIZON = 4 ticks.
- Per robot, per tick: sense (SENSOR_RANGE 2) → receive broadcasts → update local peer table (stale dropped at 6) → auction / claim / release → detect conflict → UBPA negotiate → plan → emit ONE move intention → broadcast PeerState.
- Parameterised, not hardcoded: BID_WINDOW 3, LOOKAHEAD 6, STALEMATE_TICKS 8, YIELD_WAIT 6, RADIO_DELAY 1 tick, battery drain 0.18 per cell — all in simulation/config.py.
- Working prototype, measured: 3 AMRs · 6 tasks · 8 Hz · 3 experiments × 5/3 seeds · 9 unit tests · mean 0.09 ms per robot step.

THE LOOP — every robot, every tick at 8 Hz. No stage has a central caller.



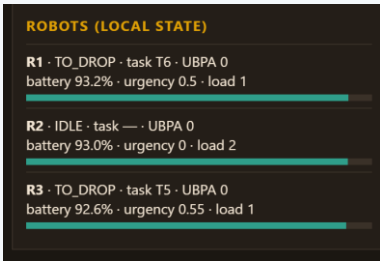
Messages on the wire: PEER_STATE · TASK_ANNOUNCE (a bulletin, never an assignment) · BID · CLAIM · RELEASE · OBSTACLE · YIELD_QUERY / YIELD_ACK / YIELD_NACK. All peer-to-peer.

RADIO (PEER PACKETS)

Robot A knows Robot B from this feed, not from a scheduler.

```
58 R3 PEER_STATE TO_DROP
58 R2 PEER_STATE IDLE
58 R1 PEER_STATE TO_DROP
57 R3 PEER_STATE TO_DROP
57 R2 PEER_STATE IDLE
57 R1 PEER_STATE TO_DROP
56 R3 PEER_STATE TO_DROP
56 R2 PEER_STATE IDLE
56 R2 CLAIM T4
56 R1 PEER_STATE TO_DROP
55 R3 PEER_STATE TO_DROP
```

PROOF — the Radio (peer packets) panel: PEER_STATE, CLAIM and OBSTACLE traffic. This feed is the ONLY channel between robots; there is no dispatcher process.



Robot state: task, battery, status, workload — held locally by each AMR and broadcast as PeerState.

FEASIBILITY AND VIABILITY



72 · 51 · 85

ticks to finish E1/E2/E3 under UBPA; stop-and-wait never finished (0/6, 1/3, 3/6 jobs inside 800 ticks).

0 deadlocks

UBPA in all three experiments. Baseline: 2 / 2 / 3 deadlocks, only 0/6 jobs done in E1.

0.09 ms

mean per-robot step time — a laptop-clock proxy for edge CPU, the only host-dependent field.

Analysis of the feasibility of the idea

- Runs anywhere Python 3.10+ runs: no GPU, no cloud, no API key, no pip install — one command starts the clock and serves the dashboard.
- Every demo knob (policy, seed, robot count, packet loss, radio delay, live cell block) is a browser control, so one terminal can run the whole demo in a hostile room.
- Deterministic under fixed seeds: re-runs reproduce every decision metric exactly; only mean_robot_step_ms moves, because it is wall-clock.
- Edge-deployable by construction: each robot is a self-contained agent with local state and a local planner, so the laptop and the future per-AMR Pi share one code path.

Potential challenges and risks

- The radio, obstacle sensing, battery drain and edge CPU time are *simulated*: no real AMR, no Wi-Fi mesh, no on-robot inference is claimed; step time is a laptop-clock proxy.
- collision_count is 2 under BOTH policies in E1 (0 vs 1 in E2), so we claim no collision-safety win — the measured win is deadlock removal and completion.
- The failure mode is measured too: at 3 ticks of radio delay the fleet still never deadlocks, but only 3/6 E1 jobs finish and intention conflicts rise to 577 — PEER_STALE_TICKS = 6 still trusts a 3-tick-stale path. Roadmap fix: age-filter peer reservations.
- No percentage speed-up is published: the baseline never finished inside the 800-tick limit, so improvement is null and comparable is false. Tuning weights live in docs/methodology.md and are re-tuned per site.

Strategies for overcoming these challenges

- Realism ladder: swap the simulated bus for UDP multicast and the grid for a real map behind the same PEER_STATE schema, so the interface survives the hardware change.
- Deterministic safety net independent of the score: after STALEMATE_TICKS the lower robot_id yields, so stale state cannot freeze the aisle even if both peers are wrong.
- Measurements are regenerated, never typed, and the live panel shows collisions, deadlocks and complete_tick side by side with the measured baseline — including the unflattering numbers.

MEASURED VS STOP-AND-WAIT — read from results/metrics.json

LIVE RUN METERS	
This panel is the current simulation, not a slide.	
collisions	2
deadlocks	0
complete_tick	156
mean_wait	0.733
distance	125.000
comm_latency	1.988
step_ms	0.127
tasks	6/6

LIVE RUN METERS — the same simulation a judge will drive: 6/6 tasks, 0 deadlocks, finished at tick 156, 0.127 ms per robot step. Note it also shows collisions = 2. We print the unflattering number rather than hide it.

MEASURED VS STOP-AND-WAIT

Loaded from results/metrics.json. Baseline vs UBPA raw values; % only if both finished.

E1 · improvement n/a (baseline incomplete)		E2 · improvement n/a (baseline incomplete)	
finished	no → yes	finished	no → yes
ticks	800 → 72	ticks	800 → 51
complete_tick	incomplete → 72	complete_tick	incomplete → 51
tasks_done	0 → 6	tasks_done	1 → 3
collisions	2 → 2	collisions	0 → 1
deadlocks	2 → 0	deadlocks	2 → 0
mean_wait	0.986 → 0.421	mean_wait	0.992 → 0.654
distance	34 → 125	distance	19 → 53
E3 · improvement n/a (baseline incomplete)			
finished	no → yes		
ticks	800 → 85		
complete_tick	incomplete → 85		
tasks_done	3 → 6		
collisions	0 → 0		
deadlocks	3 → 0		
mean_wait	0.957 → 0.369		
distance	104 → 161		

Every value here is regenerated by python scripts/run_experiments.py — improvement is null, comparable is false.

IMPACT AND BENEFITS

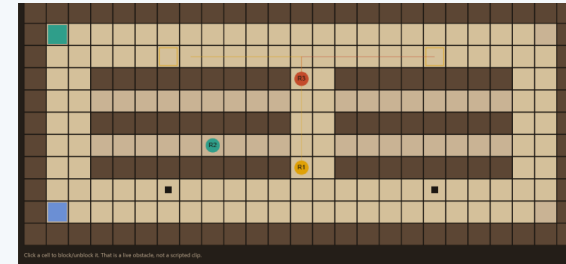
Potential impact on the target audience

- Warehouse / 3PL operators running AMR fleets: the aisle keeps working when the fleet-management server, the Wi-Fi or one robot fails — the coordination lives inside the robots, so there is nothing central left to go down.
- Indian MSME warehouses, cold-chain and 3PL hubs (the target here is a Hyderabad 3PL floor): narrow rack aisles are exactly where a single occupancy model breaks, and where one 2-robot standoff costs a whole shift.
- Fleet integrators and AMR vendors: conflict resolution is one policy file behind the same broadcast schema, so UBPA drops into an existing stack without a dispatcher rewrite.
- The operator on the floor: click a cell or press BLOCK AISLE and the fleet re-plans around the real obstruction instead of waiting for a central map to be updated.

Benefits of the solution (social, economic, environmental, etc.)

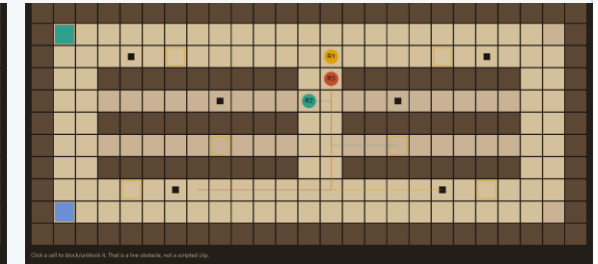
- Economic — throughput: 6/6 tasks vs 0/6 in E1, and a blocked aisle recovered in 85 ticks instead of never. Deadlock is downtime, so removing it is the return.
- Economic — deployment: standard library only, no GPU, no cloud, no licence, no service to configure; one edge computer per robot.
- Operational — degradation is bounded and measured: at 40% packet loss the fleet still finishes 6/6 with 0 deadlocks (93–125 ticks vs 72), and at 3 ticks of delay it starves but still never deadlocks.
- Social — a safer floor: an explicit, explainable right-of-way rule plus a deterministic stalemate fallback replaces ad-hoc blocking, and every decision is visible on the peer radio feed.
- Environmental — indirect: less stalled idle running per completed task; total distance rises only because the baseline barely moves while UBPA actually finishes the work.
- Research value — the harness measures deadlocks, collisions, wait fraction, distance, communication latency and edge CPU time against a stop-and-wait baseline, so any future policy is judged on one scale.

UBPA — tick 44



3 of 6 tasks done, 0 deadlocks, 0 stalemates. R1 and R3 met head-on and resolved it through a turnout.

STOP-AND-WAIT — tick 44



0 of 6 tasks done, 2 deadlocks. R1 and R3 meet head-on and resolved it through a turnout.

Same seed, same map, same tick — only the conflict policy differs

- E1 throughput, 5 seeds: UBPA finishes at tick 72 with 6/6 tasks and 0 deadlocks; stop-and-wait hits the 800-tick limit with 0/6 tasks and 2 deadlocks.
- E3, aisle blocked at tick 16: UBPA recovers in 85 ticks with 6/6 tasks and 0 deadlocks; the baseline strands at 3/6 with 3 deadlocks.
- Disclosed, not hidden: collisions are 2 under both policies in E1 (0 vs 1 in E2). We claim deadlock removal and completion — not a collision-safety win.

RESEARCH AND REFERENCES



Details / Links of the reference and research work

- Problem statement SIH26123 — Edge-AI Based Distributed Fleet Coordination for Autonomous Mobile Robots in Smart Warehouses · Bharat Electronics Limited · Software · Theme: Robotics and Drones — sih.gov.in
- Stern, Sturtevant, Felner, Koenig, Ma, Walker, Li, Atzmon, Cohen, Kumar, Boyarski, Barták (2019), “Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks”, arXiv:1906.08291 — the MAPF framing our planner sits in.
- Silver (2005), “Cooperative Pathfinding”, AIIDE-05, pp. 117–122 — the original decentralized conflict-resolution formulation our yield stage builds on.
- Sharon, Stern, Felner, Sturtevant (2015), “Conflict-Based Search for Optimal Multi-Agent Pathfinding”, Artificial Intelligence 219:40–66 — the centralized optimum we deliberately do NOT depend on.
- VDA 5050 — the industry interface standard (MQTT/JSON) between AMRs/AGVs and a master control system; it is the boundary our edge deployment would speak at, not the coordination we use.
- Warehouse AMR fleet practice — our own problem analysis of the central-dispatcher failure mode, and the Hyderabad 3PL layout (narrow rack aisles, two highways, depot and drop) used as the test floor.

Method, our own research artefacts, and how to reproduce every number

- Decentralization is enforced by test, not by promise: `python -m unittest tests.test_mandatory` runs 9 tests, including the two forbidden patterns — “no central scheduler” and “no hardcoded winner”.
- Every number in this deck is regenerated by `python scripts/run_experiments.py` into `results/metrics.json` (tick limit 800, fixed seeds, means across seeds). The dashboard only reads that file; no metric string is typed into the UI.
- Reproduce it: `python scripts/run_demo.py` (live 8 Hz, default <http://127.0.0.1:8765/>) · `python scripts/run_experiments.py` · `python -m unittest tests.test_mandatory`.
- Our artefacts: `docs/architecture.md` (what is centralized vs not), `docs/methodology.md` (planner, auction cost, UBPA weights, baseline), `docs/evaluation.md` (per-seed tables, known limitations), `docs/mandatory_requirements.md` (10 musts + 4 forbidden patterns).
- No improvement percentage is claimed: `results/metrics.json` carries improvement: null and comparable: false, because the baseline never finished inside the tick limit.
- Every screenshot in this deck is the running prototype at 127.0.0.1:8765 — no mock-ups and no re-drawn UI.